

Problem 0

Work through the Xcelium Tutorial to complete the RTL simulation of the example design of the conventional Arithmetic Logic Unit (alu_conv.v) provided in the Assignment folder, using the testbench file (alu_conv_test.v) also provided in the Assignment folder. Then, work through the Genus Tutorial using the same alu design.

Problem 1

Minimizing Two-level Logic Functions Using the QM Method

Simplify the following Boolean functions using the Quine-McCluskey algorithm. You need to (1) find out all the prime implicants (2) find out a minimum cover using techniques from the class: remember to first attempt to reduce the covering table by **trying to identify essential columns and using row/column elimination rules**. If a cyclic core is obtained, then use branch and bound.

(a)

$$f(a, b, c, d) = S(0, 5, 6, 10, 11, 13) + d(4, 8, 14)$$

Answer:

- Step 1: Write minterms in binary

Minterms:

$$\begin{aligned} 0 &= 0000, & 5 &= 0101, & 6 &= 0110, \\ 10 &= 1010, & 11 &= 1011, & 13 &= 1101 \end{aligned}$$

Don't-cares:

$$4 = 0100, \quad 8 = 1000, \quad 14 = 1110$$

- Step 2: Sort by the counts of 1's (include don't-cares)

Group 0 (0 ones): 0000

Group 1 (1 one): 1000, 0100

Group 2 (2 ones): 0101, 0110, 1010

Group 3 (3 ones): 1011, 1101, 1110

- Step 3: First-step combinations (differ by one bit)

$$\begin{aligned}
 &\begin{cases} 0000, 0100 \Rightarrow 0-00 \\ 0000, 1000 \Rightarrow -000 \end{cases} \quad (\text{Group } 0 \rightarrow 1) \\
 &\begin{cases} 1000, 1010 \Rightarrow 10-0 \\ 0100, 0101 \Rightarrow 010- \\ 0100, 0110 \Rightarrow 01-0 \end{cases} \quad (\text{Group } 1 \rightarrow 2) \\
 &\begin{cases} 0101, 1101 \Rightarrow -101 \\ 0110, 1110 \Rightarrow -110 \\ 1010, 1011 \Rightarrow 101- \\ 1010, 1110 \Rightarrow 1-10 \end{cases} \quad (\text{Group } 2 \rightarrow 3)
 \end{aligned}$$

- Step 4: No two dash-terms above can be combined further (differ in more than one position).

Prime Implicants:

$$\{0-00, -000, 10-0, 010-, 01-0, -101, -110, 101-, 1-10\}$$

- Step 5: Prime implicant chart (only required minterms need to be covered)

	0	5	6	10	11	13
0-00 ($a'c'd'$)	×					
-000 ($b'c'd'$)	×					
10-0 ($ab'd'$)				×		
010- ($a'bc'$)		×				
01-0 ($a'bd'$)			×			
-101 ($bc'd$)		×				×
-110 (bcd')			×			
101- ($ab'c$)				×	×	
1-10 (acd')				×		

- Step 6: Identify essential columns and reduce the table

- Column 11 is covered only by 101- ($ab'c$) $\Rightarrow ab'c$ is **essential**.
- Column 13 is covered only by -101 ($bc'd$) $\Rightarrow bc'd$ is **essential**.

After selecting the two essential PIs $ab'c$ and $bc'd$, the covered minterms are $\{10, 11, 5, 13\}$. Remaining to cover: $\{0, 6\}$.

$$0 : \{0-00 (a'c'd'), -000 (b'c'd')\}$$

$$6 : \{01-0 (a'bd'), -110 (bcd')\}$$

Pick one from each line to complete a minimum cover.

- Minimum cover and final SOP expression

Since both options in each pair produce equivalent coverage, any of the following canonical minimal forms are acceptable:

$$f_1 = ab'c + bc'd + a'c'd' + a'bd'$$

$$f_2 = ab'c + bc'd + a'c'd' + bcd'$$

$$f_3 = ab'c + bc'd + b'c'd' + a'bd'$$

$$f_4 = ab'c + bc'd + b'c'd' + bcd'$$

- **Final Answer:**

$$f(a, b, c, d) = ab'c + bc'd + a'c'd' + a'bd'$$

(b)

$$f(a, b, c) = S(0, 2, 3, 4, 5, 7)$$

Answer:

- Step 1: Write minterms in binary

Minterms:

$$0 = 000, \quad 2 = 010, \quad 3 = 011, \quad 4 = 100, \quad 5 = 101, \quad 7 = 111$$

- Step 2: Sort by the counts of 1's

Group 0 (0 ones): 000

Group 1 (1 one): 010, 100

Group 2 (2 ones): 011, 101

Group 3 (3 one): 111

- Step 3: First-step combinations (differ by one bit)

$$\begin{cases} 000, 010 \Rightarrow 0-0 \\ 000, 100 \Rightarrow -00 \end{cases} \quad (\text{Group } 0 \rightarrow 1)$$

$$\begin{cases} 010, 011 \Rightarrow 01- \\ 100, 101 \Rightarrow 10- \end{cases} \quad (\text{Group } 1 \rightarrow 2)$$

$$\begin{cases} 011, 111 \Rightarrow -11 \\ 101, 111 \Rightarrow 1-1 \end{cases} \quad (\text{Group } 2 \rightarrow 3)$$

- Step 4: No two dash-terms above can be combined further (differ in more than one position).

- Step 5: Prime implicant chart ($f(a, b, c) = \sum m(0, 2, 3, 4, 5, 7)$)

minterm	$P_1(0X0)$	$P_2(X00)$	$P_3(01X)$	$P_4(10X)$	$P_5(X11)$	$P_6(1X1)$
000	✓	✓				
010	✓		✓			
100		✓		✓		
011			✓		✓	
101				✓		✓
111					✓	✓

- Step 6: Row/column reduction and dominance

- First, consider the branch **Branch 1: excluding P_1** . Temporarily remove column P_1 ; in the chart, minterms 000 and 010 are highlighted in red to indicate that they will be handled later.
- Apply the dominance rule: $P_4(10X)$ dominates $P_2(X00)$, and $P_5(X11)$ dominates $P_3(01X)$. Therefore, columns P_2 and P_3 can be eliminated.

minterm	$P_4(10X)$	$P_5(X11)$	$P_6(1X1)$
100	✓		
011		✓	
101	✓		✓
111		✓	✓

- Step 7: Calculation and explanation of the lower bound

In the problem, each column can cover at most two rows. Therefore, at least three columns are required to cover all six rows. Thus, the problem has the lower bound of

$$L = 3$$

- Step 8: Branch-and-Bound decision

- From Step 7, the lower bound indicates that no feasible cover can use fewer than three prime implicants.
- A valid cover that achieves this bound is:

$$\boxed{P_1(0X0), P_4(10X), P_5(X11)}$$

where P_1 covers $\{000, 010\}$, P_4 covers $\{100, 101\}$, and P_5 covers $\{011, 111\}$.

- $\Rightarrow$ With these **three** prime implicants, all minterms are fully covered, and since **3** = lower bound, **no further branching is necessary** (the minimum solution is confirmed).

- Minimum cover and final SOP expression

$$\boxed{f(a, b, c) = a'c' + ab' + bc}$$

(corresponding to P_1, P_4, P_5).

Problem 2

Value Representations For each of the following fill the table with the corresponding information. Note that Base = {Decimal, Binary, Octal, Hexadecimal,etc.}

	Signed/Unsigned	Width	Base	Equivalent Integer Value
8'b00001000	Unsigned	8	Binary	8
16'hABCD	Unsigned	16	Hexadecimal	43981
4'sb1110	Signed	4	Binary	-2
4'd12	Unsigned	4	Decimal	12
4'b1100	Unsigned	4	Binary	12
8'shFF	Signed	8	Hexadecimal	-1
8'sb00011100	Signed	8	Binary	28
12'h2A8	Unsigned	12	Hexadecimal	680
6'sb111101	Signed	6	Binary	-3
12'o3456	Unsigned	12	Octal	1838

Problem 3

Verilog Module Statement

Write a Verilog module in RTL style for the combinational circuit in the figure shown below in Figure 1. Write a testbench and simulate the Verilog code using Excelium for all input combinations to match the truth table shown below. Include all Verilog codes and screenshots of the simulation output.

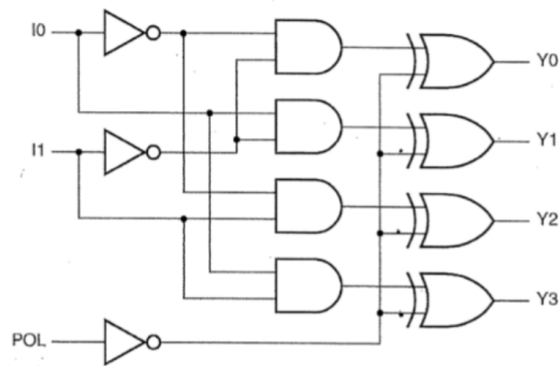


Figure 1: A 2-to-4 decoder with output-polarity control

Answer:

```

1  `timescale 1ns/1ps
2
3  module decoder_2to4(I, POL, Y);
4      input [1:0] I;      // 2-bit input
5      input POL;          // polarity control
6      output [3:0] Y;     // 4-bit output
7
8      wire [1:0] inv_I;
9      wire [3:0] and_out;
10
11     assign inv_I = ~I;
12     assign and_out[0] = inv_I[0] & inv_I[1];
13     assign and_out[1] = I[0] & inv_I[1];
14     assign and_out[2] = inv_I[0] & I[1];
15     assign and_out[3] = I[0] & I[1];
16
17     assign Y = and_out ^ {4{~POL}};
18 endmodule
19

```

(a) 2-to-4 decoder Verilog RTL module.

```

1  tb_problem3.v
2  `timescale 1ns/1ps
3  module tb_decoder_2to4;
4
5      reg [1:0] I;
6      reg POL;
7      wire [3:0] Y;
8
9      decoder_2to4 dut (
10         .I (I),
11         .POL(POL),
12         .Y (Y)
13     );
14
15     integer vec;
16     initial begin
17         // VCD dump for waveform
18         $dumpfile("decoder_2to4_tb.vcd");
19         $dumpvars(0, tb_decoder_2to4);
20         // Header
21         $display(" time | POL I1 I0 || Y3 Y2 Y1 Y0");
22         $display("-----");
23
24         // Sweep all inputs {POL,I} = 3'b000 .. 3'b111
25         for (vec = 0; vec < 8; vec = vec + 1) begin
26             {POL, I} = vec[2:0];
27             #1; // allow signal settle
28             $display("%4t | %0b %0b %0b || %0b %0b %0b %0b",
29                 $time, POL, I[1], I[0], Y[3], Y[2], Y[1], Y[0]);
30         end
31     end
32 endmodule

```

(b) Testbench used for truth table generation for 2-to-4 decoder.

Figure 2: RTL design and corresponding testbench for the 2-to-4 decoder.

```

xcelium> run
time | POL I1 I0 || Y3 Y2 Y1 Y0
-----
1000 | 0 0 0 || 1 1 1 0
2000 | 0 0 1 || 1 1 0 1
3000 | 0 1 0 || 1 0 1 1
4000 | 0 1 1 || 0 1 1 1
5000 | 1 0 0 || 0 0 0 1
6000 | 1 0 1 || 0 0 1 0
7000 | 1 1 0 || 0 1 0 0
8000 | 1 1 1 || 1 0 0 0
xmsim: *W,RNQUIE: Simulation is complete.
xcelium> exit

```

Figure 3: Simulation output according to the 2-to-4 decoder with its testbench

Problem 4

Verilog Module Statement

Write RTL-style Verilog module corresponding to the full adder circuit in Figure 4. Then, instantiate multiple copies of it to create a structural model for a 4-bit ripple carry adder structure of Figure 5. Create a testbench for the adder, apply the following input combinations

- (a) A=1, B=3
- (b) A=11, B=9
- (c) A=7, B=13
- (d) A=15, B=1

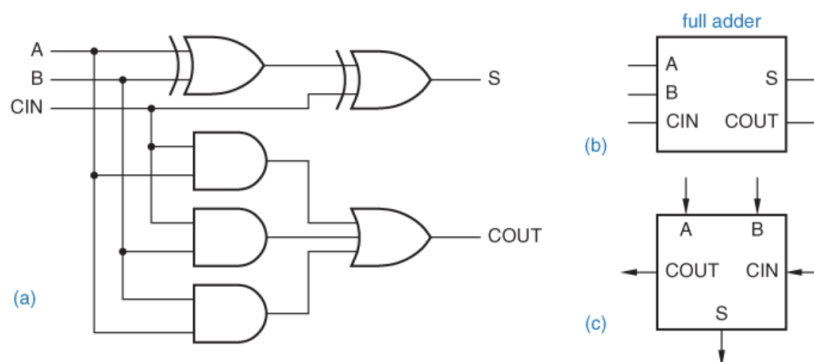


Figure 4: Full adder: (a) gate-level logic diagram; (b) logic symbol; (c) alternate logic symbol suitable for cascading.

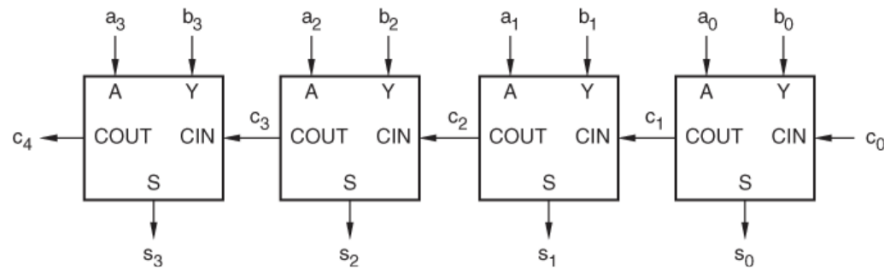


Figure 5: A 4-bit ripple adder.

Answer:

```

problem4_FA.v
1  `timescale 1ns/1ps
2
3  module FA(A, B, Cin, S, Cout);
4      input A, B, Cin;
5      output S, Cout;
6
7      wire xorAB, andAB, andAC, andBC;
8
9      assign xorAB = A ^ B;
10     assign andAB = A & B;
11     assign andAC = A & Cin;
12     assign andBC = B & Cin;
13
14     assign S = xorAB ^ Cin;
15     assign Cout = andAB | andAC | andBC;
16
17 endmodule

```

(a) 1 bit Full Adder Verilog RTL module.

```

problem4_RCA4.v
1  `timescale 1ns/1ps
2  module RCA4(A, B, Cin, S, Cout);
3      input [3:0] A, B;
4      input Cin;
5      output [3:0] S;
6      output Cout;
7
8      wire [3:0] C;
9
10     FA FA0(A[0], B[0], Cin, S[0], C[0]);
11     FA FA1(A[1], B[1], C[0], S[1], C[1]);
12     FA FA2(A[2], B[2], C[1], S[2], C[2]);
13     FA FA3(A[3], B[3], C[2], S[3], C[3]);
14     assign Cout = C[3];
15 endmodule

```

(b) 4 bit Ripple Carry Adder Verilog RTL module.

Figure 6: RTL design for Full Adder and 4 bit ripple Carry Adder verilog code


```

tb_problem4_RCA4.v
1  `timescale 1ns/1ps
2  module tb_RCA4;
3
4      reg [3:0] A, B;
5      reg      Cin;
6      wire [3:0] S;
7      wire      Cout;
8
9      RCA4 dut (.A(A),.B(B),.Cin(Cin),
10             .S(S),.Cout(Cout));
11
12     integer i;
13     reg [3:0] A_vec [0:3];
14     reg [3:0] B_vec [0:3];
15
16     initial begin
17         A_vec[0] = 4'd1; B_vec[0] = 4'd3; // (a)
18         A_vec[1] = 4'd11; B_vec[1] = 4'd9; // (b)
19         A_vec[2] = 4'd7; B_vec[2] = 4'd13; // (c)
20         A_vec[3] = 4'd15; B_vec[3] = 4'd1; // (d)
21
22         Cin = 0;
23
24         $display(" time | Cin |   A   +   B   |   S   Cout ");
25         $display("-----");
26
27         for (i = 0; i < 4; i = i + 1) begin
28             A = A_vec[i];
29             B = B_vec[i];
30             #1;
31             $display("%4t |  %b  | %2d(%b) + %2d(%b) | %2d(%b)  %b",
32                     $time, Cin, A, A, B, B, S, S, Cout);
33         end
34     end
35 endmodule

```

Figure 7: Testbench used for truth table generation for 4 bit RCA.

```

xcelium> run
time | Cin |   A   +   B   |   S   Cout
-----
1000 |  0  |  1(0001) +  3(0011) |  4(0100)  0
2000 |  0  | 11(1011) +  9(1001) |  4(0100)  1
3000 |  0  |  7(0111) + 13(1101) |  4(0100)  1
4000 |  0  | 15(1111) +  1(0001) |  0(0000)  1
xmsim: *W,RNQUIE: Simulation is complete.
xcelium> exit

```

Figure 8: Simulation output according to the 4 bits RCA with its testbench